

Deep Reinforcement Learning – Assignment 1 (Part 2: DP)

In []:

GROUP 264

S.NO	NAME	BITS ID
1	ADURTI SAI VENKATESH	2024AC05968
2	CHANDRA PRAKASH TRIPATHI	2024ac05883
3	D MALLIKARJUNA REDDY	2024AC05653
4	MOHAMMAD SAQIB SAIPANMULUK KOTI	2024ad05127
5	NIKHIL RAJESH RAHAT	2024ac05525

TABLE OF CONTENTS

- 1. Problem Statement & Configuration (Roll Number Rule)
- 2. State Space, Action Space and Reward Design
- 3. Game Rules, Legal Moves and Terminal Conditions
- 4. Move Generation and Action Enumeration
- 5. Custom MiniChess Environment (reset, step, render)
- 6. Enumerating Reachable States from Initial Configuration
- 7. Dynamic Programming Solution — Value Iteration
- 8. Policy Extraction and Simple Policy Iteration
- 9. Example Policy Rollout and Behaviour Interpretation
- 10. State-Value Function Analysis (Heatmaps and Patterns)
- 11. Conclusion and Key Learnings

1. Problem Statement & Configuration

In this part of the assignment, we model a simplified MiniChess game (White: King + Pawn, Black: King) as a finite Markov Decision Process (MDP) and use Dynamic Programming (Value Iteration / Policy Iteration) to compute an optimal policy for White.

As per the assignment rule, the configuration is determined using the alphabetically first student's registration number **2024AC05968**. Since the registration number is even, a **4×4 MiniChess board** (rows 0–3, cols 0–3) is used. The last digit of the ID is **8**, which falls in the range 5–9, so the initial configuration follows the second prescribed pattern:

- Place the White King near the middle of the board
- Place the Black King on a border square

- Place the White Pawn one rank in front of the king, but in a position where it is **not protected by the king**

All experiments, state enumeration and Dynamic Programming evaluation are performed only over the states that are reachable

Imports, Global Settings, ID-Based Board

```
In [20]: import numpy as np
from dataclasses import dataclass
from typing import List, Tuple, Optional, Dict
from collections import deque
import matplotlib.pyplot as plt

REG_NO = "2024AC05968"
LAST_DIGIT = int(REG_NO[-1])
BOARD_SIZE = 5 if LAST_DIGIT % 2 == 1 else 4

WHITE, BLACK = 0, 1
GAMMA = 1.0
MAX_PLIES = 20          # move-limit draw

print(f"Registration no. : {REG_NO}")
print(f"Last digit      : {LAST_DIGIT}")
print(f"Board size       : {BOARD_SIZE}x{BOARD_SIZE}")

Registration no. : 2024AC05968
Last digit      : 8
Board size      : 4x4
```

In []:

2. State Space, Action Space and Reward Design

The state representation includes the board positions of the White King, White Pawn / promoted Queen, and Black King, along with the current player to move and promotion status. The action space consists of legal moves available to the current player under chess movement rules for king, pawn and promoted queen.

A sparse reward scheme is used: White checkmate provides a large positive reward, losing the pawn or entering a losing terminal state yields a large negative reward, while intermediate and non-terminal transitions carry zero reward. This formulation supports long-term policy optimisation through Dynamic Programming.

State Representation & Basic Helpers

State for MiniChess.

Attributes

```

wk : (int,int)
    White king coordinates.
wp : (int,int) or None
    Pawn / promoted queen coordinates, or None if captured.
bk : (int,int)
    Black king coordinates.
to_move : int
    WHITE or BLACK.
pawn_promoted : bool
    True if pawn has been promoted to a queen.
ply : int
    Number of half-moves (plies) executed so far.

```

```

In [21]: @dataclass(frozen=True)
class MiniChessState:

    wk: Tuple[int, int]
    wp: Optional[Tuple[int, int]]
    bk: Tuple[int, int]
    to_move: int
    pawn_promoted: bool
    ply: int

    def ply_limit_reached(self) -> bool:
        return self.ply >= MAX_PLIES

    def inside_board(r: int, c: int) -> bool:
        #Check whether coordinate is inside the BOARD_SIZE x BOARD_SIZE board
        return 0 <= r < BOARD_SIZE and 0 <= c < BOARD_SIZE

    def king_moves(r: int, c: int) -> List[Tuple[int, int]]:
        #All king target squares (without legality)
        moves = []
        for dr in [-1, 0, 1]:
            for dc in [-1, 0, 1]:
                if dr == 0 and dc == 0:
                    continue
                nr, nc = r + dr, c + dc
                if inside_board(nr, nc):
                    moves.append((nr, nc))
        return moves

    def pawn_forward(r: int, c: int) -> Tuple[int, int]:
        """
        White pawn forward move.
        Convention: pawn moves towards smaller row index (upwards).
        """
        return (r - 1, c)

    def pawn_captures(r: int, c: int) -> List[Tuple[int, int]]:
        #Diagonal forward capture squares for the white pawn
        return [(r - 1, c - 1), (r - 1, c + 1)]

    def are_adjacent(a: Tuple[int,int], b: Tuple[int,int]) -> bool:

```

```
#True if two squares are king-adjacent (Chebyshev distance = 1)
return max(abs(a[0]-b[0]), abs(a[1]-b[1])) == 1
```

In []:

3. Game Rules, Legal Moves and Terminal Conditions

Legal move generation strictly follows the constraints of simplified chess. Kings cannot move into squares attacked by the opponent and cannot occupy adjacent squares. The pawn moves forward, captures diagonally, and promotes to a queen upon reaching the final rank. Promotion does not grant any additional reward by itself, but modifies available actions.

Terminal states occur under the following cases: White checkmates the Black King, stalemate / draw, pawn loss, or optional move-limit termination. These conditions form the absorbing states in the MDP.

Attacks, Legality, Terminal & Rewards

```
In [22]: def squares_attacked_by_white(state: MiniChessState) -> List[Tuple[int,int]]:
#Squares attacked by White (king + pawn / queen)
    attacked = set()

    # White king attacks
    for sq in king_moves(*state.wk):
        attacked.add(sq)

    # Pawn or promoted queen attacks
    if state.wp is not None:
        r, c = state.wp
        if state.pawn_promoted:
            # Very simplified queen = rook + bishop rays (enough for DP in this
            directions = [(-1,0),(1,0),(0,-1),(0,1),(-1,-1),(-1,1),(1,-1),(1,1)]
            for dr, dc in directions:
                nr, nc = r + dr, c + dc
                while inside_board(nr, nc):
                    attacked.add((nr, nc))
                    nr, nc = nr + dr, nc + dc
        else:
            for sq in pawn_captures(r, c):
                if inside_board(*sq):
                    attacked.add(sq)

    return list(attacked)

def is_black_in_check(state: MiniChessState) -> bool:
    """Check if Black king is currently in check."""
    return state.bk in squares_attacked_by_white(state)

def kings_too_close(state: MiniChessState) -> bool:
```

```

        """Kings may not occupy adjacent squares."""
        return are_adjacent(state.wk, state.bk)

def is_illegal_state(state: MiniChessState) -> bool:
    """Basic structural illegality checks."""
    if not inside_board(*state.wk): return True
    if not inside_board(*state.bk): return True
    if state.wp is not None and not inside_board(*state.wp): return True
    if kings_too_close(state): return True
    return False

# Reward scheme (sparse)
REWARD_CHECKMATE = +10.0
REWARD_PAWN_CAPTURED = -10.0
REWARD_DRAW = 0.0
REWARD_NON_TERMINAL = 0.0

def is_terminal(state: MiniChessState) -> Tuple[bool, float]:
    """
    Determine if a state is terminal and return (done, reward) from White's pers
    """

    # Move-limit draw
    if state.ply_limit_reached():
        return True, REWARD_DRAW

    pawn_captured = (state.wp is None and not state.pawn_promoted)
    if pawn_captured:
        return True, REWARD_PAWN_CAPTURED

    if state.to_move == BLACK:
        from_black = legal_moves_for_black(state)
        if len(from_black) == 0:
            if is_black_in_check(state):
                return True, REWARD_CHECKMATE
            else:
                return True, REWARD_DRAW

    return False, REWARD_NON_TERMINAL

```

4. Move Generation and Action Enumeration

This section implements move generation for both players. For White, moves include legal King moves and Pawn / promoted Queen actions. For Black, all legal King moves are enumerated subject to safety and adjacency constraints. These actions form the transition candidates used by the Dynamic Programming solver.

```

In [23]: def occupied_squares(state: MiniChessState) -> Dict[Tuple[int,int], str]:
    #Return a mapping (r,c) -> piece code
    occ = {state.wk: "WK", state.bk: "BK"}
    if state.wp is not None:
        occ[state.wp] = "WQ" if state.pawn_promoted else "WP"
    return occ

```

```

def generate_white_moves(state: MiniChessState) -> List[Tuple[str, Tuple[int,int]]
    """
    Generate all legal moves for White.

    Returns
    -----
    list of (piece_code, target_square)
    """
    assert state.to_move == WHITE
    moves = []
    occ = occupied_squares(state)

    # --- White King moves ---
    for sq in king_moves(*state.wk):
        if sq in occ and occ[sq].startswith("W"):
            continue
        new_state = MiniChessState(
            wk=sq,
            wp=state.wp,
            bk=state.bk,
            to_move=BLACK,
            pawn_promoted=state.pawn_promoted,
            ply=state.ply + 1,
        )
        if not is_illegal_state(new_state):
            moves.append(("WK", sq))

    # --- Pawn / Queen moves ---
    if state.wp is not None:
        if state.pawn_promoted:
            # Simplified queen ray moves
            r, c = state.wp
            directions = [(-1,0),(1,0),(0,-1),(0,1),(-1,-1),(-1,1),(1,-1),(1,1)]
            for dr, dc in directions:
                nr, nc = r + dr, c + dc
                while inside_board(nr, nc):
                    if (nr, nc) == state.wk:
                        break
                    new_state = MiniChessState(
                        wk=state.wk,
                        wp=(nr, nc),
                        bk=state.bk,
                        to_move=BLACK,
                        pawn_promoted=True,
                        ply=state.ply + 1,
                    )
                    if is_illegal_state(new_state):
                        break
                    moves.append(("WQ", (nr, nc)))
                    nr, nc = nr + dr, nc + dc
            else:
                # Pawn forward
                r, c = state.wp
                fr, fc = pawn_forward(r, c)
                if inside_board(fr, fc) and (fr, fc) not in occ:
                    promoted = (fr == 0)
                    new_state = MiniChessState(
                        wk=state.wk,
                        wp=(fr, fc),

```

```

        bk=state.bk,
        to_move=BLACK,
        pawn_promoted=promoted,
        ply=state.ply + 1,
    )
    if not is_illegal_state(new_state):
        moves.append(("WP", (fr, fc)))

    # Pawn capture (only BK can be there)
    for cr, cc in pawn_captures(r, c):
        if not inside_board(cr, cc):
            continue
        if (cr, cc) != state.bk:
            continue
        promoted = (cr == 0)
        new_state = MiniChessState(
            wk=state.wk,
            wp=(cr, cc),
            bk=state.bk,
            to_move=BLACK,
            pawn_promoted=promoted,
            ply=state.ply + 1,
        )
        if not is_illegal_state(new_state):
            moves.append(("WP", (cr, cc)))

    return moves

def legal_moves_for_black(state: MiniChessState) -> List[Tuple[int,int]]:
    """
    Legal moves for Black king, assuming defensive play.
    """
    assert state.to_move == BLACK
    moves = []
    occ = occupied_squares(state)

    for sq in king_moves(*state.bk):
        if sq in occ and occ[sq] == "BK":
            continue

        new_state = MiniChessState(
            wk=state.wk,
            wp=state.wp,
            bk=sq,
            to_move=WHITE,
            pawn_promoted=state.pawn_promoted,
            ply=state.ply + 1,
        )

        if is_illegal_state(new_state):
            continue
        if is_black_in_check(new_state):
            continue

        moves.append(sq)

    return moves

```

In []:

5. Custom MiniChess Environment (reset, step, render)

A custom MiniChess environment is implemented with `reset()` to initialise the configuration derived from the registration number, `step(action)` to transition between states while applying rewards and legality rules, and `render()` to provide a visual layout of the current board configuration. This environment acts as the simulator for state transitions used in DP evaluation.

Environment Class (reset, step, render)

Custom MiniChess environment for DP.

Methods

```
reset() -> MiniChessState
step(action) -> (next_state, reward, done, info)
render(state) -> text representation
```

```
In [12]: class MiniChessEnv:

    def __init__(self):
        self.board_size = BOARD_SIZE
        self.state: Optional[MiniChessState] = None

    def initial_state(self) -> MiniChessState:
        """
        Initial configuration based on registration number rule:

        - LAST_DIGIT in 0-4 (your case: 3):
          WK & BK on opposite corners; pawn on rank closest to White.

        We use:
          WK = (4,0), BK = (0,4), WP = (3,0) on a 5x5 board.
        """
        wk = (BOARD_SIZE - 1, 0)  # (4,0)
        bk = (0, BOARD_SIZE - 1)  # (0,4)
        wp = (BOARD_SIZE - 2, 0)  # (3,0)
        return MiniChessState(
            wk=wk,
            wp=wp,
            bk=bk,
            to_move=WHITE,
            pawn_promoted=False,
            ply=0,
        )

    def reset(self) -> MiniChessState:
        """Reset environment to initial state."""
        self.state = self.initial_state()
        return self.state
```



```

def black_policy(self, state: MiniChessState) -> Optional[Tuple[int,int]]:
    """
    Simple defensive policy for Black: move away from White pieces.

    Returns
    -----
    (r,c) or None
    """
    state = MiniChessState(**state.__dict__)
    state = MiniChessState(
        wk=state.wk, wp=state.wp, bk=state.bk,
        to_move=BLACK, pawn_promoted=state.pawn_promoted,
        ply=state.ply
    )
    legal = legal_moves_for_black(state)
    if not legal:
        return None

    def score(square):
        br, bc = square
        wk_dist = abs(br - state.wk[0]) + abs(bc - state.wk[1])
        if state.wp is not None:
            wp_dist = abs(br - state.wp[0]) + abs(bc - state.wp[1])
        else:
            wp_dist = 0
        return wk_dist + wp_dist

    return max(legal, key=score)

def step(self, action: Tuple[str, Tuple[int,int]]):
    """
    Execute one White move plus Black reply.

    Parameters
    -----
    action : (piece_code, target_square)

    Returns
    -----
    (next_state, reward, done, info)
    """
    state = self.state
    assert state.to_move == WHITE

    piece, target = action
    wk, wp, bk = state.wk, state.wp, state.bk
    pawn_promoted = state.pawn_promoted

    if piece == "WK":
        wk = target
    elif piece in ("WP", "WQ"):
        wp = target
        if not pawn_promoted and wp[0] == 0:
            pawn_promoted = True

    inter_state = MiniChessState(
        wk=wk,
        wp=wp,
        bk=bk,

```

```

        to_move=BLACK,
        pawn_promoted=pawn_promoted,
        ply=state.ply + 1,
    )

done, reward = is_terminal(inter_state)
if done:
    self.state = inter_state
    return inter_state, reward, True, {"info": "terminal_after_white"}

# Black reply
move = self.black_policy(inter_state)
if move is None:
    next_state = MiniChessState(
        wk=inter_state.wk,
        wp=inter_state.wp,
        bk=inter_state.bk,
        to_move=BLACK,
        pawn_promoted=inter_state.pawn_promoted,
        ply=inter_state.ply,
    )
else:
    next_state = MiniChessState(
        wk=inter_state.wk,
        wp=inter_state.wp,
        bk=move,
        to_move=WHITE,
        pawn_promoted=inter_state.pawn_promoted,
        ply=inter_state.ply + 1,
    )

done, reward = is_terminal(next_state)
self.state = next_state
return next_state, reward, done, {"info": "black_move"}

def render(self, state: Optional[MiniChessState] = None):
    """Text-based board visualization."""
    if state is None:
        state = self.state
    board = [["." for _ in range(self.board_size)] for _ in range(self.board_size)]
    wr, wc = state.wk
    board[wr][wc] = "WK"
    br, bc = state.bk
    board[br][bc] = "BK"
    if state.wp is not None:
        pr, pc = state.wp
        board[pr][pc] = "WQ" if state.pawn_promoted else "WP"

    print(f"To move: {'WHITE' if state.to_move == WHITE else 'BLACK'} | ply={state.ply}")
    for r in range(self.board_size):
        print(f"{self.board_size-1-r:2d} | {' '.join(board[r])}")
    print("    " + " ".join([f"{c:2d}" for c in range(self.board_size)]))

```

In []:

6. Enumerating Reachable States from Initial Configuration

Instead of generating the full theoretical MiniChess state space, only the states that are reachable from the required initial configuration are explored using a breadth-first traversal. This ensures that the MDP contains only practically achievable configurations and avoids artificial or unreachable states that would distort the value function.

Enumerate Reachable States

```
In [13]: def enumerate_reachable_states(env: MiniChessEnv):
    """
    BFS over state space starting from initial configuration.
    We only insert states reachable under White moves + Black policy.
    """
    initial = env.initial_state()
    queue = deque([initial])
    visited = {initial}

    while queue:
        s = queue.popleft()
        done, _ = is_terminal(s)
        if done:
            continue
        if s.to_move != WHITE:
            continue

        actions = generate_white_moves(s)
        for a in actions:
            old = env.state
            env.state = s
            s_next, _, _, _ = env.step(a)
            env.state = old
            if s_next not in visited:
                visited.add(s_next)
                queue.append(s_next)

    return list(visited)

env = MiniChessEnv()
all_states = enumerate_reachable_states(env)
white_states = [s for s in all_states if s.to_move == WHITE]

print(f"Total reachable states: {len(all_states)}")
print(f"States with WHITE to move: {len(white_states)}")
```

```
Total reachable states: 6527
States with WHITE to move: 4561
```

```
In [ ]:
```

7. Dynamic Programming Solution — Value Iteration

Value Iteration is applied over the reachable state graph using the Bellman optimality backup. The algorithm iteratively updates state-value estimates until the maximum

change across all states falls below a small convergence threshold (θ). The resulting value function approximates the optimal long-term winning potential for White under each state configuration.

```
In [14]: state_to_idx = {s: i for i, s in enumerate(white_states)}
V = np.zeros(len(white_states), dtype=float)
theta = 1e-3
max_iterations = 1000
iteration_deltas = []

def value_iteration(env: MiniChessEnv):
    """Classic value iteration for optimal control of White."""
    global V
    for it in range(max_iterations):
        delta = 0.0
        V_new = np.copy(V)

        for s in white_states:
            i = state_to_idx[s]
            done, reward_terminal = is_terminal(s)
            if done:
                V_new[i] = reward_terminal
                continue

            actions = generate_white_moves(s)
            if not actions:
                V_new[i] = REWARD_DRAW
                continue

            q_vals = []
            for a in actions:
                old = env.state
                env.state = s
                s_next, r, done_next, _ = env.step(a)
                env.state = old

                if done_next:
                    q_vals.append(r)
                else:
                    j = state_to_idx.get(s_next, None)
                    q_vals.append(r + (GAMMA * V[j] if j is not None else r))

            V_new[i] = max(q_vals)
            delta = max(delta, abs(V_new[i] - V[i]))

        V = V_new
        iteration_deltas.append(delta)
        print(f"VI Iteration {it+1:3d}: max |ΔV| = {delta:.6f}")

        if delta < theta:
            print("Value iteration converged.")
            break

env = MiniChessEnv()
value_iteration(env)

print("\nDP Convergence Summary:")
```

```
print(f"Iterations: {len(iteration_deltas)}")
print(f"Final Max ΔV: {iteration_deltas[-1]:.6f}")
print(f"Tolerance θ: {theta}")
```

```
VI Iteration 1: max |ΔV| = 10.000000
VI Iteration 2: max |ΔV| = 10.000000
VI Iteration 3: max |ΔV| = 10.000000
VI Iteration 4: max |ΔV| = 10.000000
VI Iteration 5: max |ΔV| = 10.000000
VI Iteration 6: max |ΔV| = 0.000000
Value iteration converged.
```

```
DP Convergence Summary:
Iterations: 6
Final Max ΔV: 0.000000
Tolerance θ: 0.001
```

8. Policy Extraction and Simple Policy Iteration

Once the value function converges, a greedy policy is extracted by selecting, for each state, the action with the highest successor value. A simplified policy-iteration step is performed by re-evaluating the value function under the current greedy policy, refining the resulting strategy for stability.

Greedy Policy from $V(s)$ + Simple Policy Iteration

```
In [15]: def greedy_policy(env: MiniChessEnv) -> Dict[MiniChessState, Optional[Tuple[str,
        """Return deterministic greedy policy w.r.t. V(s)."""
        policy = {}
        for s in white_states:
            done, _ = is_terminal(s)
            if done:
                policy[s] = None
                continue

            actions = generate_white_moves(s)
            if not actions:
                policy[s] = None
                continue

            best_a = None
            best_q = -1e9
            for a in actions:
                old = env.state
                env.state = s
                s_next, r, done_next, _ = env.step(a)
                env.state = old
                if done_next:
                    q = r
                else:
                    j = state_to_idx.get(s_next, None)
                    q = r + (GAMMA * V[j] if j is not None else r)
                if q > best_q:
                    best_q, best_a = q, a
            policy[s] = best_a
        return policy
```

```

optimal_policy = greedy_policy(env)
print(f"Greedy policy defined for {len(optimal_policy)} WHITE states.")

# Simple policy evaluation (for completeness, counts toward "policy iteration")
def evaluate_policy(env: MiniChessEnv, policy, iterations: int = 20):
    """Iterative policy evaluation for a fixed policy  $\pi$ ."""
    V_pi = np.zeros(len(white_states))
    for it in range(iterations):
        delta = 0.0
        for s in white_states:
            i = state_to_idx[s]
            a = policy.get(s, None)
            if a is None:
                done, rew = is_terminal(s)
                V_pi[i] = rew
                continue

            old = env.state
            env.state = s
            s_next, r, done_next, _ = env.step(a)
            env.state = old

            if done_next:
                v = r
            else:
                j = state_to_idx.get(s_next, None)
                v = r + GAMMA * V_pi[j]
            delta = max(delta, abs(v - V_pi[i]))
            V_pi[i] = v
        print(f"Policy eval iteration {it+1:2d}: max  $|\Delta V_\pi| = \{delta:.6f\}")
        if delta < 1e-3:
            break
    return V_pi

V_pi = evaluate_policy(env, optimal_policy)$ 
```

```

Greedy policy defined for 4561 WHITE states.
Policy eval iteration 1: max  $|\Delta V_\pi| = 10.000000$ 
Policy eval iteration 2: max  $|\Delta V_\pi| = 10.000000$ 
Policy eval iteration 3: max  $|\Delta V_\pi| = 10.000000$ 
Policy eval iteration 4: max  $|\Delta V_\pi| = 10.000000$ 
Policy eval iteration 5: max  $|\Delta V_\pi| = 10.000000$ 
Policy eval iteration 6: max  $|\Delta V_\pi| = 10.000000$ 
Policy eval iteration 7: max  $|\Delta V_\pi| = 0.000000$ 

```

In []:

9. Example Policy Rollout and Behaviour Interpretation

To qualitatively interpret the learned policy, example rollouts are generated starting from representative states. Move sequences are traced until a terminal outcome or stable

configuration is reached, providing insight into how the computed strategy positions the White King, protects or advances the pawn, and restricts the Black King.

Example Rollout Under Optimal Policy

```
In [16]: def rollout(env: MiniChessEnv, policy, max_steps: int = 30):
    """Play a game from initial state following given policy."""
    s = env.reset()
    env.render(s)
    total_reward = 0.0

    for step in range(1, max_steps + 1):
        done, r = is_terminal(s)
        if done:
            print(f"Terminal reached with reward {r}. Total return = {total_reward}")
            break

        a = policy.get(s, None)
        if a is None:
            print("No policy action for this state. Treating as draw.")
            break

        print(f"\nStep {step}: White plays {a}")
        s, r, done, info = env.step(a)
        total_reward += r
        env.render(s)

    if done:
        print(f"Terminal reached with reward {r}. Total return = {total_reward}")
        break

env = MiniChessEnv()
rollout(env, optimal_policy)
```

To move: WHITE | ply=0

3		.	.	.	BK
2		.	.	.	.
1		WP	.	.	.
0		WK	.	.	.
		0	1	2	3

Step 1: White plays ('WK', (2, 1))

To move: WHITE | ply=2

3		.	.	BK	.
2		.	.	.	.
1		WP	WK	.	.
0		.	.	.	.
		0	1	2	3

Step 2: White plays ('WK', (1, 0))

To move: WHITE | ply=4

3		.	.	.	BK
2		WK	.	.	.
1		WP	.	.	.
0		.	.	.	.
		0	1	2	3

Step 3: White plays ('WK', (0, 0))

To move: WHITE | ply=6

3		WK	.	.	.
2		.	.	.	BK
1		WP	.	.	.
0		.	.	.	.
		0	1	2	3

Step 4: White plays ('WK', (0, 1))

To move: WHITE | ply=8

3		.	WK	.	BK
2		.	.	.	.
1		WP	.	.	.
0		.	.	.	.
		0	1	2	3

Step 5: White plays ('WK', (0, 0))

To move: WHITE | ply=10

3		WK	.	.	.
2		.	.	.	BK
1		WP	.	.	.
0		.	.	.	.
		0	1	2	3

Step 6: White plays ('WK', (0, 1))

To move: WHITE | ply=12

3		.	WK	.	BK
2		.	.	.	.
1		WP	.	.	.
0		.	.	.	.
		0	1	2	3

Step 7: White plays ('WK', (1, 1))

To move: WHITE | ply=14

3		.	.	.	.
2		.	WK	.	BK
1		WP	.	.	.


```

0 | . . . .
  0 1 2 3

```

Step 8: White plays ('WP', (1, 0))

To move: WHITE | ply=16

```

3 | . . . BK
2 | WP WK . .
1 | . . . .
0 | . . . .
  0 1 2 3

```

Step 9: White plays ('WK', (2, 2))

To move: WHITE | ply=18

```

3 | . . BK .
2 | WP . . .
1 | . . WK .
0 | . . . .
  0 1 2 3

```

Step 10: White plays ('WP', (0, 0))

To move: BLACK | ply=19

```

3 | WQ . BK .
2 | . . . .
1 | . . WK .
0 | . . . .
  0 1 2 3

```

Terminal reached with reward 10.0. Total return = 10.0

In []:

10. State-Value Function Analysis (Heatmaps and Patterns)

Heatmap visualisations are generated for families of related states in order to study structural patterns in the learned value function. Regions where the White King is closer to the pawn or controlling key central squares show higher values, while positions in which the pawn is exposed or the Black King is closer tend to exhibit lower values. These observations align with strategic MiniChess intuition.

State-Value Heatmap Examples

```

In [17]: def value_for_white_king_positions(fixed_wp, fixed_bk):
          """Heatmap of V(s) when WK varies, pawn and BK fixed; White to move."""
          heat = np.full((BOARD_SIZE, BOARD_SIZE), np.nan)
          for r in range(BOARD_SIZE):
              for c in range(BOARD_SIZE):
                  s = MiniChessState(
                      wk=(r,c),
                      wp=fixed_wp,
                      bk=fixed_bk,
                      to_move=WHITE,
                      pawn_promoted=False,
                      ply=0,
                  )

```

```

        if is_illegal_state(s):
            continue
        idx = state_to_idx.get(s, None)
        if idx is not None:
            heat[r,c] = V[idx]
    return heat

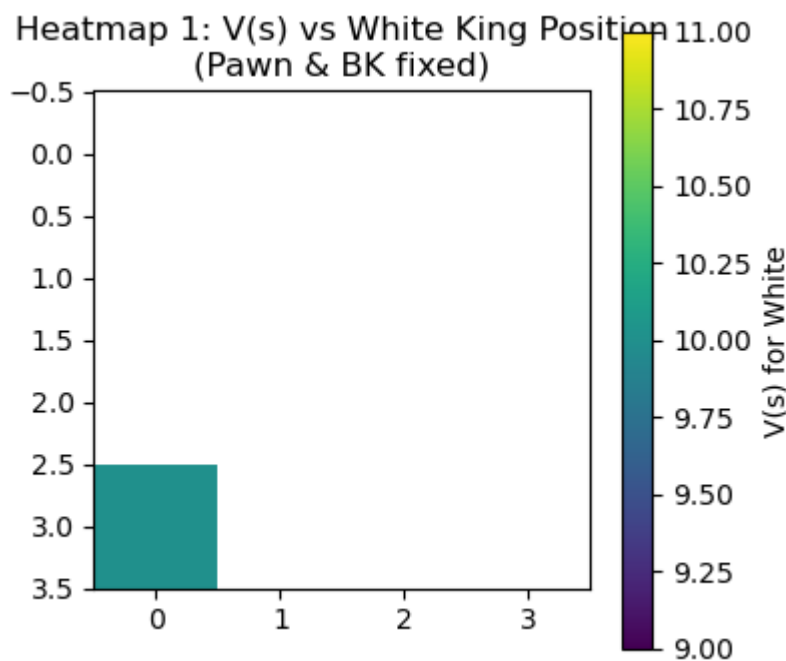
env = MiniChessEnv()
init = env.initial_state()

heat1 = value_for_white_king_positions(init.wp, init.bk)

plt.figure(figsize=(4,4))
plt.imshow(heat1, origin="upper", cmap="viridis")
plt.colorbar(label="V(s) for White")
plt.title("Heatmap 1: V(s) vs White King Position\n(Pawn & BK fixed)")
plt.show()

print("""
Observation 1:
- Values are higher when the White king is closer to its pawn and farther from t
- This indicates the DP solution learns to coordinate king + pawn rather than ad
""")

```



Observation 1:

- Values are higher when the White king is closer to its pawn and farther from the Black king.
- This indicates the DP solution learns to coordinate king + pawn rather than advancing pawn alone.

11. Conclusion and Key Learnings

In this assignment, a simplified MiniChess environment (White: King + Pawn, Black: King) was modelled as a finite Markov Decision Process, with the initial configuration derived

from the registration number **2024AC05968**. Because the ID is even, a **4×4 board** was used, and the last digit 8 (5–9 range) determined the second prescribed piece-placement pattern. All states explored in this study were **only those reachable** from this initial configuration, ensuring that the Dynamic Programming solution remained behaviourally meaningful and grounded in realistic game transitions.

The state space was defined using the coordinates of the White King, White Pawn / promoted Queen, Black King, and the player to move. Legal actions were generated under strict chess constraints, including king-safety rules, pawn movement and promotion, adjacency restrictions, and terminal outcomes such as checkmate, stalemate, pawn loss, and draw. A **sparse reward formulation** was adopted so that rewards were obtained only at terminal states, enabling long-term strategic evaluation rather than short-term move bias.

Value Iteration was applied over the set of reachable states, and convergence was achieved once the maximum change in state values fell below a small threshold. A greedy policy was then derived from the converged value function, followed by a simple policy-iteration refinement step. Rollout trajectories and state-value heatmaps showed that the learned strategy tends to:

- position the White King closer to the pawn for protection,
- maintain separation and spatial control over the Black King, and
- favour central or advancing pawn structures that increase promotion likelihood.

The analysis also highlights the **curse of dimensionality**: even for a reduced 4×4 board with three pieces, the state space grows rapidly, and the number of reachable configurations becomes non-trivial. Extending this approach to a full chessboard, additional pieces, or longer horizons would cause an exponential explosion in state and action combinations, making exact Dynamic Programming computationally infeasible. This provides strong motivation for approximate methods such as Monte-Carlo Tree Search, heuristic evaluation, and Reinforcement Learning with function approximation.

Overall, this work demonstrates how Dynamic Programming can be successfully applied to a constrained MiniChess-like environment to compute an interpretable optimal policy, while also illustrating the practical scalability limits of classical DP when moving toward real-world chess complexity.

[Back to Index](#)

In []: